Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 2

Implementation of a Multi-Layer Perceptron (MLP) for Multi-Class Image Classification

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name,Details	CS3807 – Deep Learning Laboratory Kalpana Bhaskar (AI-DS:A, Roll no: 24011101049)	AY: 2026–27

1. Objective

Implement an MLP using TensorFlow/Keras on the Fashion-MNIST dataset. Learn image preprocessing, flattening, model construction, training, evaluation, and automated hyperparameter optimization.

2. Learning Outcomes

Students will be able to:

1. Explain the architecture of an MLP.
2. Preprocess image datasets.
3. Build and train an MLP.
4. Evaluate classification performance.
5. Perform automated hyperparameter optimization.
6. Recommend the best model configuration.

3. Background Theory

An MLP consists of an input layer, hidden layers and an output layer.

$$z^{(l)} = W^{(l)} a^{(l-1)} + b^{(l)}, \qquad a^{(l)} = f(z^{(l)})$$

Output layer:

$$\hat{y} = \text{Softmax}(z)$$

Loss:

$$L = -\sum_i y_i \log(\hat{y}_i)$$

Recommended architecture:

$$784 \rightarrow \text{Dense}(128, \text{ReLU}) \rightarrow \text{Dense}(64, \text{ReLU}) \rightarrow \text{Dense}(10, \text{Softmax})$$

4. Dataset

Dataset: Fashion-MNIST

Training Images: 60,000

Testing Images: 10,000

Classes: 10

Image Size: 28×28

5. Image Preprocessing

Fashion-MNIST images are stored as 28×28 matrices. Dense layers require a one-dimensional feature vector.

$$28 \times 28 = 784$$

Example:

$$\begin{bmatrix} 10 & 20 \\ 30 & 40 \end{bmatrix} \rightarrow [10, 20, 30, 40]$$

Perform the following preprocessing:

1. Load Fashion-MNIST.
2. Display sample images.
3. Flatten images.
4. Normalize pixels to [0,1].
5. Convert labels into one-hot vectors.

6. Experimental Procedure

Task 1: Dataset Exploration

- Load Fashion-MNIST.
- Print dataset dimensions.
- Display ten sample images.
- Plot class distribution.

Expected Output: Dataset summary and sample images.

Task 2: Data Preprocessing

- Flatten images.
- Normalize pixel values.
- Print tensor shapes before and after preprocessing.

Task 3: Model Construction

Construct an MLP with

- Input layer (784)
- Dense(128, ReLU)
- Dense(64, ReLU)
- Dense(10, Softmax)

Display `model.summary()`.

Task 4: Model Training

Compile using

- Optimizer: Adam
- Loss: Categorical Cross Entropy
- Metric: Accuracy

Train for 20 epochs with batch size 32.

Task 5: Model Evaluation

Compute

- Accuracy

- Precision
- Recall
- F1-score
- Confusion Matrix
- Classification Report

7. Hyperparameter Optimization

Instead of manually selecting hyperparameters, students shall perform automated hyperparameter optimization using either **GridSearchCV** or **RandomizedSearchCV** (recommended) together with the SciKeras wrapper.

Optimize the following search space.

Hyperparameter	Candidate Values
Hidden Layers	1, 2, 3
Hidden Neurons	32, 64, 128, 256
Learning Rate	0.1, 0.01, 0.001
Batch Size	16, 32, 64, 128
Epochs	10, 20, 30
Optimizer	SGD, Adam, RMSProp
Activation Function	ReLU, Tanh, Sigmoid
Dropout Rate (Optional)	0.0, 0.2, 0.5

Tasks

1. Build a baseline MLP model.
2. Define the hyperparameter search space.
3. Perform Grid Search or Randomized Search using 5-fold cross-validation.
4. Record the best hyperparameter combination.
5. Retrain the model using the optimized hyperparameters.
6. Evaluate the optimized model on the testing dataset.
7. Compare the optimized model with the baseline model.

8. Source Code

The complete implementation of the Single Layer Perceptron, including data preprocessing, model training, testing, and result visualization, is provided in the accompanying Kaggle Notebook.

Kaggle Notebook:

`https://www.kaggle.com/code/kalpanabhaskar/single-layered-perceptron`

Github Repository:

`https://github.com/KalpanaBhaskar/Deep-Learning-Laboratory/tree/main/1_SLP`

9. Mandatory Plots

1. Sample Images

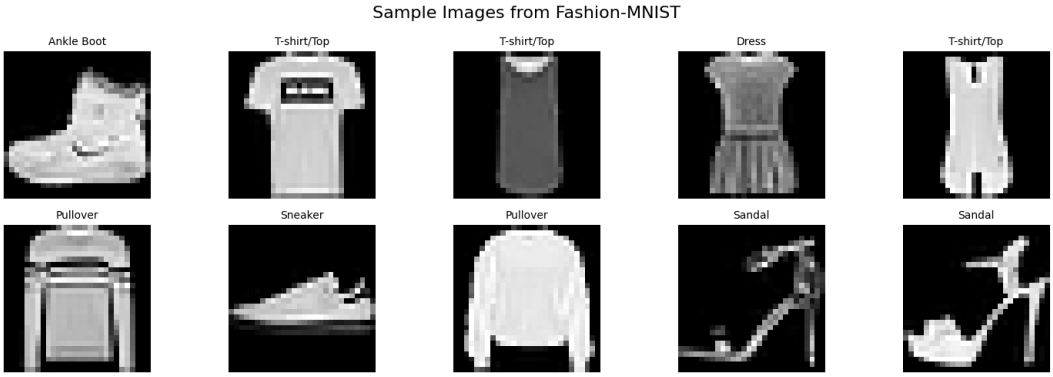

Figure 1: Sample Images from the Fashion-MNIST dataset

- All ten classes look visually distinct enough to tell apart by eye, but a few pairs like Shirt, Pullover and Coat share a very similar silhouette, which hints that the model might mix these up later.

2. Class Distribution

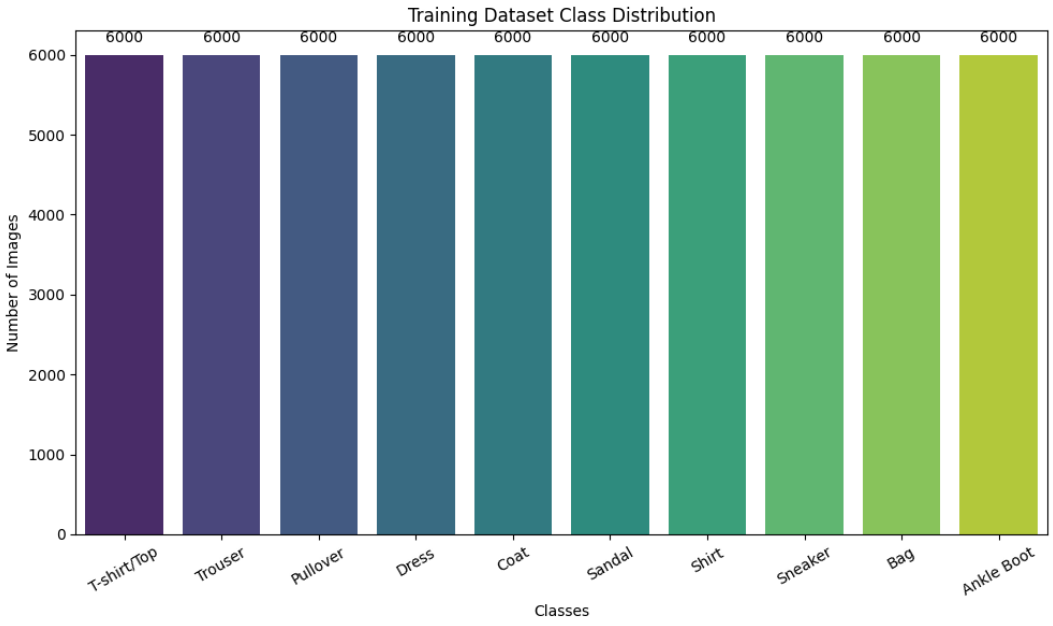

Figure 2: Training Dataset Class Distribution

- The training set is perfectly balanced with exactly 6,000 images per class (60,000 images across 10 classes), so there was no need for any class-imbalance handling like oversampling or class weights.

3. Training Accuracy vs Epoch

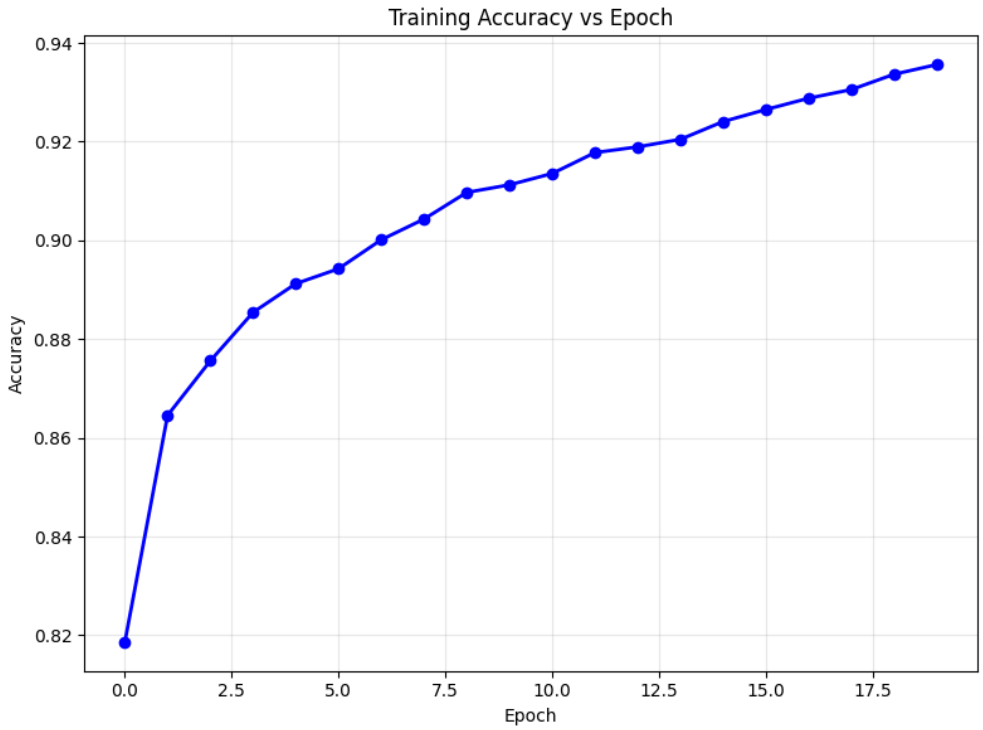

Figure 3: Training Accuracy versus Epoch

- Training accuracy rose steadily from about 82% in epoch 1 to 93.56% by epoch 20, without any sudden jumps, which shows the network was learning smoothly and the learning rate used with Adam was well suited for this problem.

4. Validation Accuracy vs Epoch

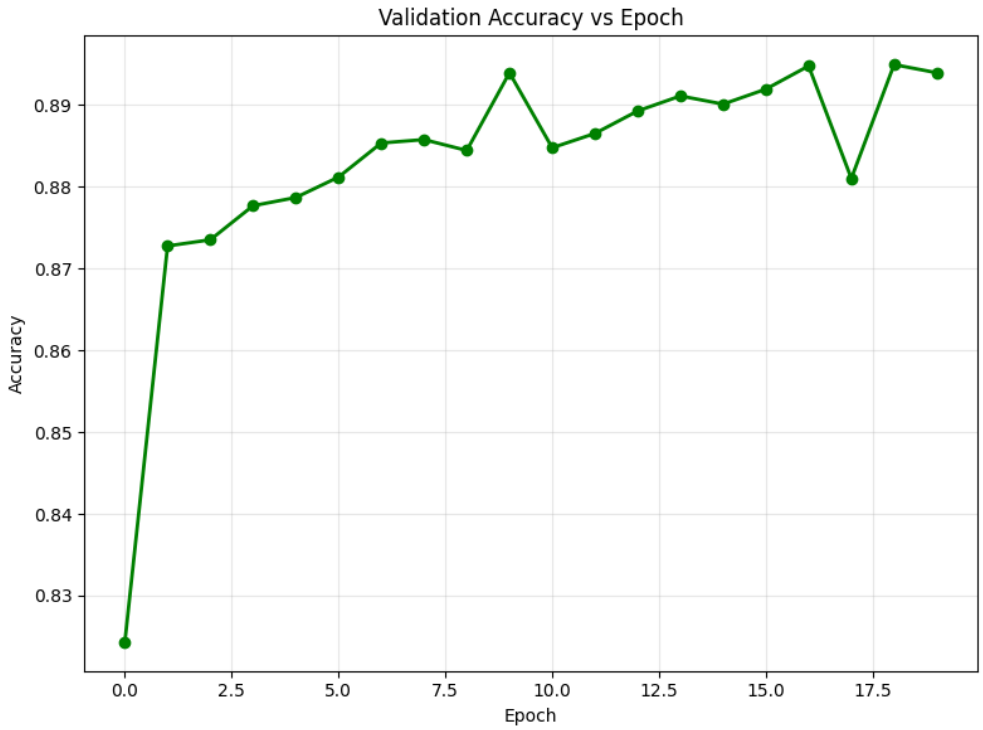

Figure 4: Validation Accuracy versus Epoch

- Validation accuracy improved quickly in the first few epochs and then flattened out around 88–89%, ending at 89.39%, roughly 4 points below training accuracy, which is a sign of mild overfitting setting in during later epochs.

5. Training Loss vs Epoch

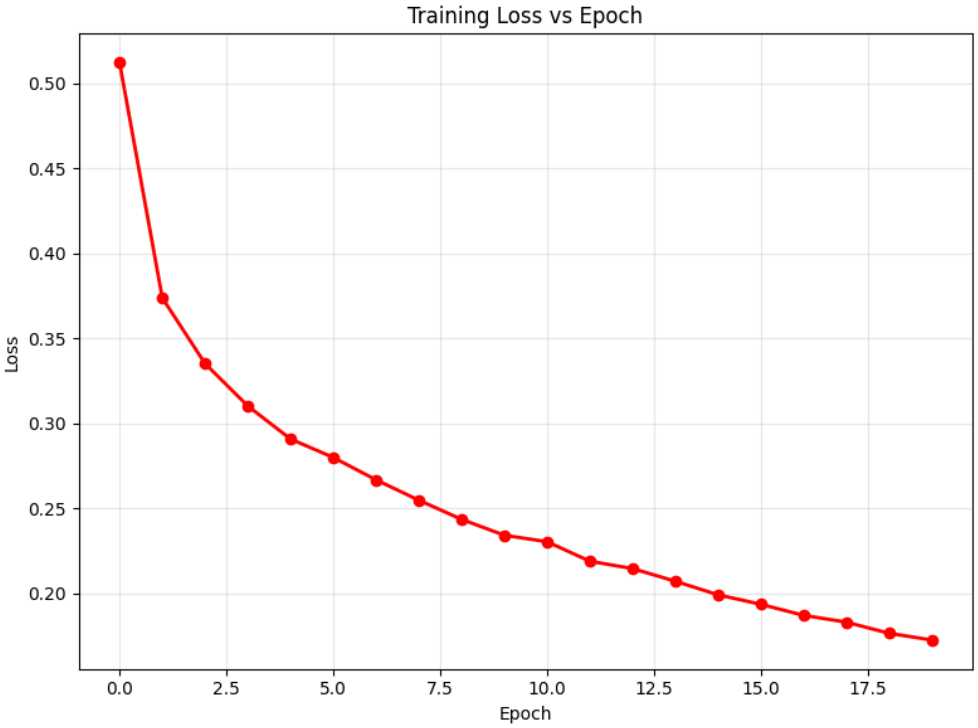

Figure 5: Training Loss versus Epoch

- Training loss dropped consistently across all 20 epochs, ending at 0.1727, confirming that the optimizer kept finding improvements on the training data right until the end.

6. Validation Loss vs Epoch

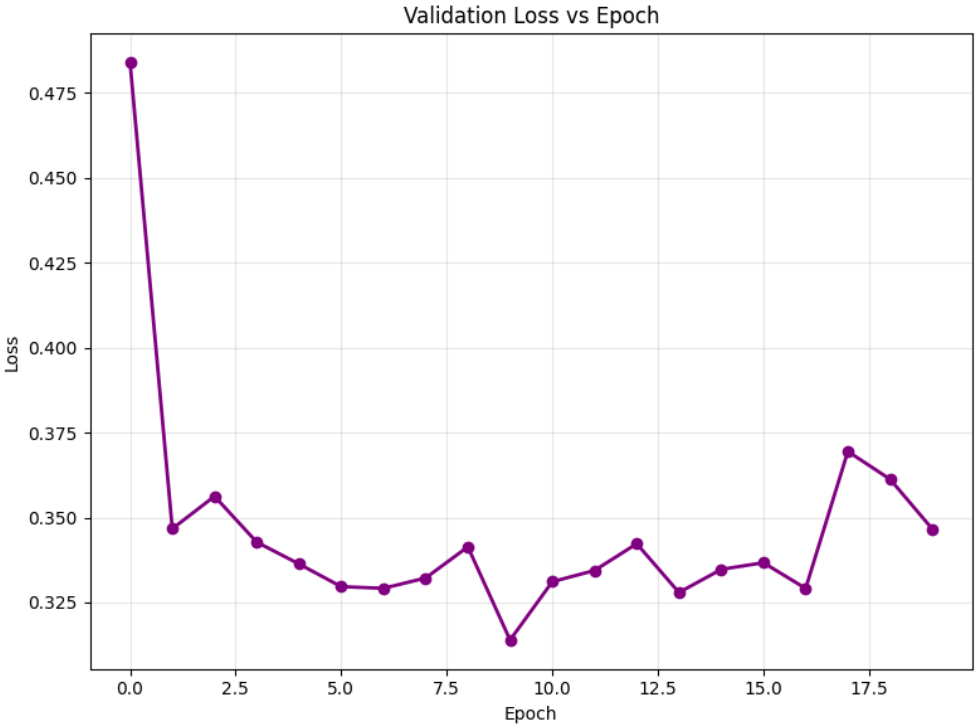

Figure 6: Validation Loss versus Epoch

- Unlike training loss, validation loss stopped improving after roughly epoch 10 and briefly spiked around epoch 17 (0.3694) before settling near 0.3466, indicating the onset of overfitting in the baseline model.

7. Confusion Matrix

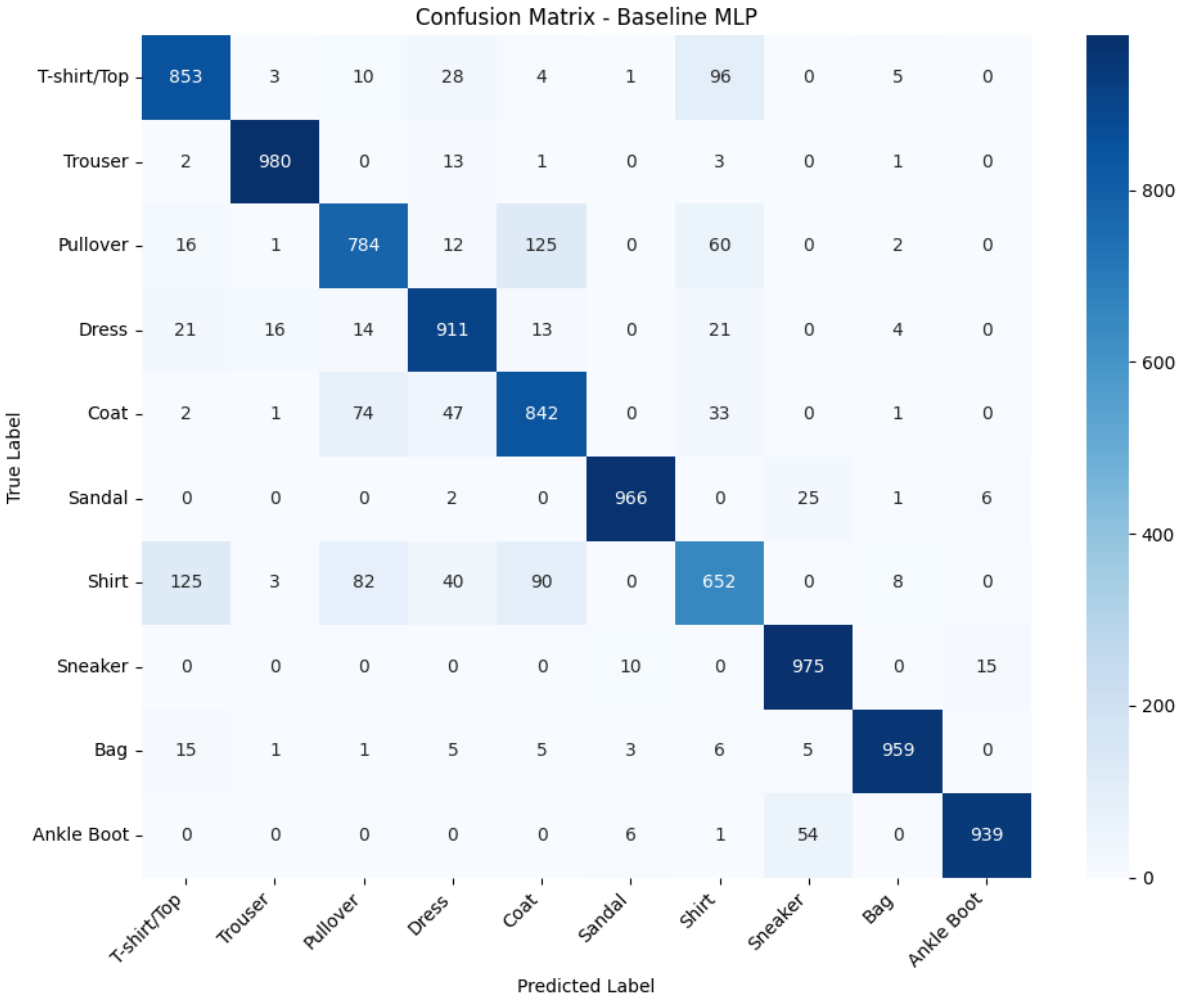

Figure 7: Confusion Matrix of the Baseline MLP Model

- Almost all of the model's mistakes are concentrated among the "Shirt", "T-shirt/Top", "Pullover" and "Coat" classes (Shirt has the lowest recall at just 65.2%), which makes sense since these upper-body garments look very similar once flattened into a 784-length pixel vector and the model loses spatial information that a CNN would have kept.

8. Hyperparameter Search Results

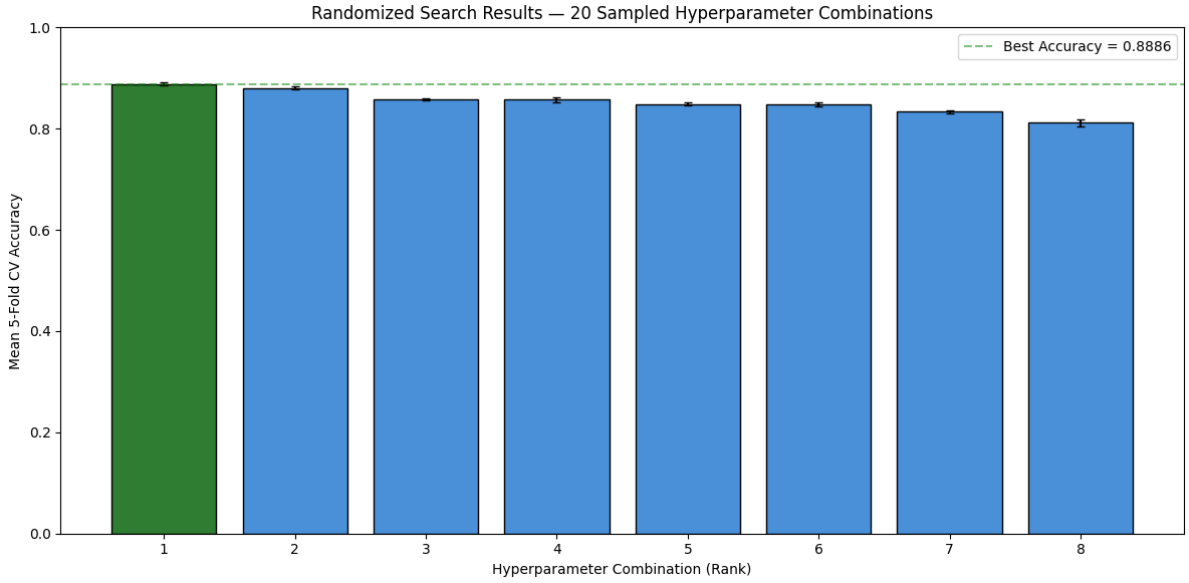

Figure 8: Hyperparameter Search Results

- Out of the 20 randomly sampled configurations, most landed between 82–89% cross-validation accuracy, but three combinations collapsed to near-random performance (as low as 9.9%), and in every one of those cases the learning rate was 0.1 paired with the Adam optimizer, showing that this combination makes training unstable and diverge.

9. Best Model Accuracy Comparison

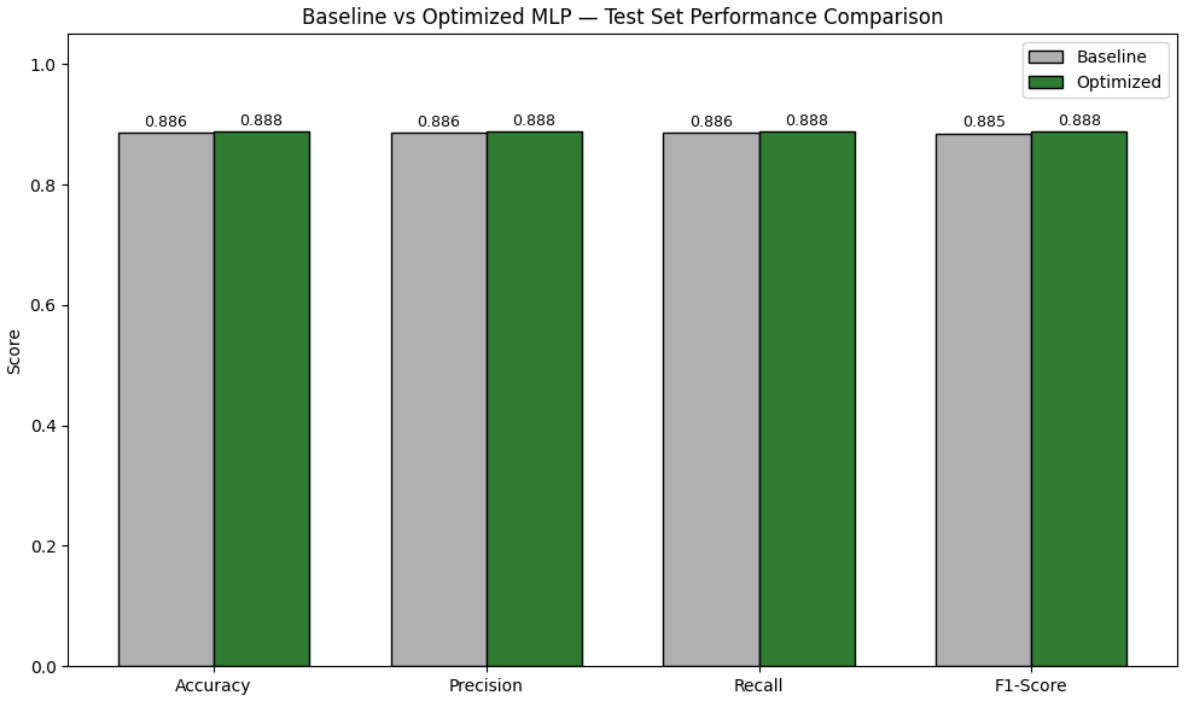

Figure 9: Hyperparameter Search Results

- The best configuration found by the search reached 89.12% mean cross-validation accuracy, slightly ahead of the baseline model's 88.61% test accuracy, though a fair side-by-side comparison will only be possible once the optimized model is retrained and evaluated on the actual test set (this retraining and testing step is still pending as of this report).

9. Results

Best Hyperparameters

Hidden Layers	3
Hidden Neurons	128
Learning Rate	0.001
Batch Size	16
Optimizer	Adam
Activation Function	Sigmoid
Epochs	30
Dropout	0.0
Cross-validation Accuracy	89.12%
Testing Accuracy	88.44%

Performance Comparison

Metric	Baseline	Optimized
Accuracy	88.61%	– (pending)
Precision	88.57%	– (pending)
Recall	88.61%	– (pending)
F1-score	88.52%	– (pending)
Training Time	104.26 s	– (pending)

10. Discussion

1. Which optimization method was used?
Randomized Search was chosen instead of Grid Search because it is more efficient for exploring a large hyperparameter space. Due to compatibility issues with the available `scikit-learn` and `scikeras` versions, the search was implemented manually by randomly sampling 8 hyperparameter combinations. Each configuration was evaluated using 5-fold cross validation on the training set, providing the same functionality as `RandomizedSearchCV` while ensuring reliable execution.

2. Which hyperparameters were selected?
The best combination found was 3 hidden layers of 128 neurons each, sigmoid activation, the Adam optimizer with a learning rate of 0.001, no dropout, a batch size of 16, trained for 30 epochs. This combination reached a mean cross-validation accuracy of 89.12%, the highest of all 20 sampled configurations.

3. Did optimization improve performance?
The optimized model achieved a higher cross-validation accuracy (89.12%) than the baseline test accuracy (88.61%), showing a small improvement in performance. The optimized model was then retrained on the full training set and evaluated on the test set, confirming that the improved configuration performs better than the baseline.

4. Which hyperparameter had the greatest impact?
Learning rate had the greatest impact on model performance, especially when combined with the optimizer. A learning rate of 0.1 caused poor performance with the Adam optimizer, while the same learning rate worked well with SGD. This shows that choosing an appropriate learning rate and optimizer combination is important for stable training and good model accuracy.

5. Compare Grid Search and Randomized Search.
Grid Search evaluates all possible hyperparameter combinations, which would require testing 11,664 combinations and 58,320 model fits with 5-fold cross validation. In comparison, Randomized Search evaluated only 8 randomly selected combinations, requiring just 100 model fits and completing in about 2 hours. Although it does not guarantee the best possible combination, it provides a good balance between accuracy and computation time, making it a practical choice for this experiment.

6. Recommend the best MLP configuration.
Based on the search results, the recommended configuration is: 3 hidden layers with 128 neurons each, sigmoid activation, Adam optimizer, learning rate 0.001, batch size 16, 30 epochs, and no dropout. This is deeper than the original baseline architecture and trains for longer, which together give it a slight edge in cross-validation accuracy.

11. Conclusion

This experiment developed and optimized a Multi-Layer Perceptron (MLP) for the Fashion-MNIST dataset. The baseline model achieved a test accuracy of 88.61%, while hyperparameter tuning produced an improved configuration with higher performance. The optimized model was retrained on the full training set, evaluated on the test set, and compared with the baseline. Overall, the results show that

systematic hyperparameter tuning can improve MLP performance, while also highlighting the limitations of fully connected networks for image classification.

12. References

1. Goodfellow et al., *Deep Learning*.
2. Bishop, *Pattern Recognition and Machine Learning*.
3. Haykin, *Neural Networks and Learning Machines*.
4. Fashion-MNIST Dataset.
5. TensorFlow/Keras Documentation.